

It Is Not the Signature: An Empirical Study of a Hidden Key-Parse Cost in JSON Web Token Validation

Jannatul Ferdous Katha^{a,c}, Tasmia Tahmid Prova^{b,c}, Md Kishor Morol^c, Tze Hui Liew^{d,*}, Dip Nandi^{c,e}

^a*Department of CSE, Bangladesh University of Professionals (BUP), Dhaka, Bangladesh*

^b*Department of CSE, Bangladesh University of Business and Technology (BUBT), Dhaka, Bangladesh*

^c*ELITE Research Lab, Queens, NY, USA*

^d*Faculty of Information Science and Technology, Multimedia University, Melaka, Malaysia*

^e*Department of Computer Science, American International University-Bangladesh (AIUB), Dhaka, Bangladesh*

Abstract

Developers routinely attribute the per-request cost of JSON Web Token (JWT) validation to cryptography. We test that assumption in `jsonwebtoken`, a widely used Node.js library. The signature primitive is 2.208 μ s of a 29.92 μ s validation; 18.04 μ s (60%) is a discarded parse, in which a string secret is first offered to the asymmetric key parser and is resolved only after that parse fails. Its price is set by a transitive dependency, the OpenSSL release beneath the runtime: issued from C, the same calls cost 394.4 μ s on OpenSSL 3.0.16 and 8.447 μ s on 3.5.8. The slow case is not confined to end-of-life runtimes: Ubuntu 24.04's packaged Node.js, linked to the system OpenSSL 3.0, pays 442 μ s per validation. In a containerised service on OpenSSL 3.0, validation accounted for 53% of the application's processor time per request. Of seven JWT libraries in four languages, only this one uses a failing parse, and only its penalty changes by an order of magnitude with the OpenSSL release. The obvious optimisation is unsafe: the parse arrived with a key-confusion security fix, and removing it naively fails the library's own tests. A fix that dispatches on the key material alone passes the full suite on four runtimes,

*Corresponding author.

Email addresses: `ferdous.katha@elitelab.ai` (Jannatul Ferdous Katha), `tasmia.prova@elitelab.ai` (Tasmia Tahmid Prova), `kishormorol@ieee.org` (Md Kishor Morol), `thliew@mmu.edu.my` (Tze Hui Liew), `dip.nandi@aiub.edu` (Dip Nandi)

behaves identically to the stock library in every case we tested, and removes the parse for every token, including forged ones; so does passing a pre-parsed key, yet none of the 327 call sites in our sample does. We derive guidance for developers, library authors and performance engineers, and release a replication package.

Keywords: Empirical study, performance analysis, software libraries, root-cause analysis, JSON Web Token, OpenSSL

1. Introduction

Performance engineering depends on attributing cost correctly. When a cost is believed to belong to one component and belongs to another, optimisation effort goes to the wrong place, and architectural remedies are adopted for problems that could have been fixed in a few lines. Empirical studies of performance bugs find that many stem from inefficient library usage and avoidable redundant work, and that the fixes are small once the cause is located (Jin et al., 2012; Selakovic and Pradel, 2016). This paper reports such a case on a path that runs on every request of many web services: the validation of JSON Web Tokens (JWTs) (Jones et al., 2015).

The usual reasoning is intuitive: validating a token means verifying a signature, signatures are cryptography, and cryptography is expensive, so the cost should be *moved* to a sidecar proxy, a gateway or dedicated hardware. The assumption has been tested before one layer down, where Coarfa et al. (2006) found RSA a minority of TLS server cost. We test it at the application layer, in `jsonwebtoken`, a widely used Node.js library, and find it wrong. A second natural explanation, that key material is converted from a string into a key object on every call, fails as well: the conversion costs 0.583 μ s.

The actual mechanism is a control-flow accident (Figure 2). Given a secret that is not already a key object, the library attempts `createPublicKey()` first and falls back to `createSecretKey()` in the exception handler. For an HMAC secret the first call cannot succeed, so every validation constructs and discards an OpenSSL parse failure, at 18.04 μ s of a 29.92 μ s call. We call this failed `createPublicKey()` call, including the exception that reports its failure, the *discarded parse*. Its price depends on which OpenSSL release performs it, by more than an order of magnitude: runtimes carrying OpenSSL 3.2 or later pay tens of microseconds, runtimes carrying the 3.0 series pay hundreds. The 3.0 series is still the system OpenSSL of long-term-support distributions,

29 and Node.js as packaged by Ubuntu 24.04 links it and pays the slow price
30 (Section 5.3); so does any image pinned to an older base layer, which is
31 common in practice (Zerouali et al., 2019).

32 For software engineering the case matters for three reasons beyond the
33 library itself. First, the cost sits in control flow that no profiler label names:
34 a parse whose failure is caught and discarded appears in profiles as OpenSSL
35 work, and the natural reading of that attribution is “cryptography”. Second,
36 its magnitude is set by a transitive dependency that application developers do
37 not choose directly and rarely record. Third, the obvious fix is unsafe. The
38 resolution block entered the library in version 9.0.0, the release that fixed an
39 RSA-to-HMAC key-confusion vulnerability (Auth0, 2022), and the key-type
40 checks implementing that fix read the type of the key object it produces;
41 removing it the obvious way fails the library’s own key-confusion tests. The
42 code that costs the time is the code that makes a security check work, and the
43 coupling is not visible from the lines being changed. The design that produces
44 the cost (resolving key material on every call, by way of a parse whose failure
45 serves as a type test) can also be named independently of `jsonwebtoken`,
46 and it predicts which of seven libraries perform a failing parse (Section 7).

47 *Research questions.*

48 **RQ1** Where does the per-request cost of token validation actually go?

49 **RQ2** What governs the magnitude of that cost?

50 **RQ3** Does it matter in a running containerised service, and how common is
51 the pattern that avoids it?

52 **RQ4** Is the defect a property of one library, or of a design choice other
53 libraries could share?

54 **RQ5** Can the cost be removed without weakening the library’s key-confusion
55 defence, and who can cause it to be paid?

56 *Contributions.* All are supported by measurements in the replication pack-
57 age (Katha et al., 2026).

- 58 1. Two independent decompositions of the validation call that locate its
59 dominant component in a discarded parse, and a split of it into the
60 OpenSSL parse, the V8 exception and the binding (Section 4).
- 61 2. A causal attribution of its magnitude to the bundled OpenSSL release,
62 with the parse reissued from C against 8 releases (Section 5).
- 63 3. Corroboration in a containerised service by cgroup CPU accounting, and
64 a survey showing the avoiding pattern is rare in public code (Section 6).

- 65 4. A comparison of seven JWT libraries in four languages showing that
66 the cost follows a design choice, not JWT validation as such, and a
67 detector-based survey of 64 plus 164 popular npm packages locating
68 the failing-parse form of that choice (Section 7).
- 69 5. An analysis of why the cost cannot be removed naively, and two fixes
70 that pass the library’s full test suite; the simpler one, dispatching on
71 the key material alone, removes the parse for every token, including
72 forged ones (Section 8).
- 73 6. Guidance for developers, library authors and performance engineers,
74 including the one-line change that removes the cost (Section 9).

75 2. Background and related work

76 *What a validation call does.* A JSON Web Token carries a header, a payload
77 and a signature over the first two (Jones et al., 2015). Validating one means
78 decoding two segments, resolving key material into a form the cryptographic
79 library accepts, and verifying the signature. The middle step, key resolution,
80 is the subject of this paper, and it is the step most often left implicit when
81 validation cost is discussed.

82 *Where the cost is usually attributed.* Measurement of layered enforcement in
83 microservices concentrates on transport and proxy layers; Zhu et al. (2023)
84 show that service-mesh sidecar overhead depends on configuration and work-
85 load rather than being a fixed tax. Work on cryptographic library performance
86 has largely stayed at the level of the primitive or the protocol exchange (Naser
87 et al., 2020; Sikeridis et al., 2020). Our result concerns neither: it is the cost
88 of getting key material *into* the library, which we could not find measured
89 directly.

90 *Performance bugs and anti-patterns.* Jin et al. (2012) found that most real-
91 world performance bugs follow a few recurring patterns, among them skippable
92 function calls, and that fixes are usually local; Nistor et al. (2013) found
93 that most are discovered by reasoning about code rather than by observing
94 symptoms. In JavaScript, Selakovic and Pradel (2016) identify inefficient API
95 usage as a leading root cause, and their later profiler (Selakovic et al., 2017)
96 targets the *order* in which a program evaluates conditions, the axis on which
97 this defect lives. The defect here is a skippable call whose failure is its only
98 observable effect.

99 *Exceptions as control flow.* Exceptions are costly in managed runtimes, mostly
100 in their construction (Ogasawara et al., 2006), and Parravicini and Mueller
101 (2021) provide the closest cost-attribution baseline for V8, though on specula-
102 tion rather than exceptions. Studies of how exceptions are used find handlers
103 that discard what they catch to be common (Cabral and Marques, 2007; Kery
104 et al., 2016; de Pádua and Shang, 2017). That literature treats a swallowed
105 exception as a maintainability smell; here it is control flow on a hot path,
106 and Section 4.1 measures how much of its cost is the exception itself.

107 *Dependencies and the npm ecosystem.* Performance changes across versions in
108 ways maintainers do not anticipate (Kaltenecker et al., 2023; Chen and Shang,
109 2017). Container images pin a dependency set implicitly (Cito et al., 2017)
110 and routinely ship outdated packages (Zerouali et al., 2019). In the npm
111 ecosystem, Staicu and Pradel (2018) show that a slow path inside widely used
112 modules reaches the web servers that depend on them, and Davis et al. (2018)
113 that on Node.js’s single event loop one slow synchronous handler delays every
114 other client. For OpenSSL 3, maintainers of a major downstream library
115 have stated that parsing and key loading regressed against 1.1.1 (Gaynor and
116 Kehrer, 2026); Section 5.2 supplies a controlled measurement for the one path
117 this paper concerns.

118 *Benchmarking pitfalls and change detection.* Our protocol follows established
119 guidance for managed-runtime benchmarking (Georges et al., 2007; Kalibera
120 and Jones, 2013; Barrett et al., 2017). Microbenchmarks go wrong in recurring
121 ways (Costa et al., 2021), developers misjudge when warm-up ends (Traini
122 et al., 2023), cloud environments degrade their reliability (Laaber et al., 2019),
123 and benchmark suites often fail to detect performance changes at all (Laaber
124 and Leitner, 2018). The attribution this paper overturns has the structure
125 Mytkowicz et al. (2009) warn about: a second variable, the bundled OpenSSL
126 version, moving unrecorded alongside the one under study.

127 *Security and performance.* Security mechanisms are routinely evaluated for
128 their performance cost: Coarfa et al. (2006) for TLS servers, and Sikeridis
129 et al. (2020) for post-quantum TLS and SSH. In that line the cost is the
130 price of the protection. Here the cost and the protection are coupled in a less
131 visible way: the code that costs the time is the code a security check depends
132 on (Section 8.3).

133 *Cryptographic APIs and JWT libraries.* Most cryptographic vulnerabilities
134 are misuses of libraries by applications (Lazar et al., 2014; Egele et al., 2013),
135 largely because the interfaces are hard to use (Nadi et al., 2016); detection has
136 been automated by specifying correct usage (Krüger et al., 2018; Rahaman
137 et al., 2019). Our finding differs: the interface admits a use that is *correct*
138 and costly only for a reason invisible at the call site. Security analyses of
139 JWT implementations consistently find key-handling defects, key confusion
140 among them (Detering et al., 2017; Xu et al., 2023); Yang et al. (2026)
141 evaluated 43 JWT libraries across 16 languages. RFC 8725 (Sheffer et al.,
142 2020) codifies the resulting practice. The library studied here has been
143 hardened against key confusion twice: CVE-2015-9235, fixed in 4.2.2 (GitHub
144 Advisory Database, 2015), and CVE-2022-23541 with two related defects,
145 fixed in 9.0.0 (Auth0, 2022). We treat the same surface as a cost site and
146 show the two views are coupled. Our survey follows guidance on mining
147 public repositories (Kalliamvakou et al., 2014).

148 2.1. What we searched for and did not find

149 Table 1 summarises the positioning. Each cell records only what this
150 section states about the work cited; it is a map of the lines of work this paper
151 connects, not an assessment of them.

152 We found no peer-reviewed study of performance regressions across the
153 OpenSSL 3.x series, none tracing the chain from base image to bundled library
154 to application performance, and none treating a correct-but-costly use of a
155 cryptographic interface as a performance defect. We state each as an absence
156 we searched for.

157 3. Methodology

158 Figure 1 gives the study’s workflow: the instruments used for each research
159 question and the tables and figures each produces. Every instrument writes
160 to committed run files, from which every number in the paper is generated
161 (Data availability).

162 3.1. Testbed

163 Unless stated otherwise, measurements were taken on one Apple-silicon
164 machine running macOS 26.5.2 (arm64), *the host*, under Node.js v26.6.0.
165 This Node.js is the Homebrew build, which links the system’s Homebrew
166 OpenSSL dynamically rather than bundling its own; the host therefore ran

Table 1: Positioning against the closest lines of work. *JWT*: studies JWT or JOSE implementations. *Crypto-API*: treats how a cryptographic interface or key material is used as a security concern. *Cost*: studies a performance cost or performance defect. *Version*: attributes a cost or defect to the version of a bundled dependency. *Coupling*: shows that removing a cost would weaken a security defence. *Breadth*: studies many systems, projects or libraries, which this paper does not (one library in depth, six others for comparison). ✓: addressed; ◦: partly (see note); -: not addressed.

Work	JWT	Crypto-API	Cost	Version	Coupling	Breadth
Detering et al. (2017); Xu et al. (2023); Yang et al. (2026)	✓	✓	–	–	–	✓
Egele et al. (2013); Nadi et al. (2016); Krüger et al. (2018); Rahaman et al. (2019)	–	✓	–	–	–	✓
Coarfa et al. (2006); Naser et al. (2020); Sikeridis et al. (2020)	–	–	✓	–	–	–
Zhu et al. (2023)	–	–	✓	–	–	–
Jin et al. (2012); Selakovic and Pradel (2016)	–	–	✓	–	–	✓
Kaltenecker et al. (2023); Chen and Shang (2017)	–	–	✓	◦ ^a	–	✓
Zerouali et al. (2019); Cito et al. (2017)	–	–	–	◦ ^b	–	✓
This paper	✓	✓	✓	✓	✓	◦ ^c

^aVersions of the system under study, not of a bundled dependency. ^bContainer images pin outdated dependency versions; no cost is measured. ^cOne library in depth; seven for comparison; a detector run over a stated frame of npm packages (Section 7).

167 OpenSSL 3.6.3, newer than the 3.5 series that official Node 26 builds bundle.
168 The version was not recorded per row for the host decomposition, and is
169 inferred from the run record (Section 10). Container measurements ran in the
170 Linux/arm64 virtual machine of Docker Desktop’s single-node Kubernetes
171 cluster (v1.32.2). The service studied in Section 6 is a Node.js microservice
172 (Express 5.1.0, `jsonwebtoken` 9.0.3) on the `node:18-alpine` image, run as
173 one replica with an Istio sidecar behind an NGINX ingress, verifying HS256

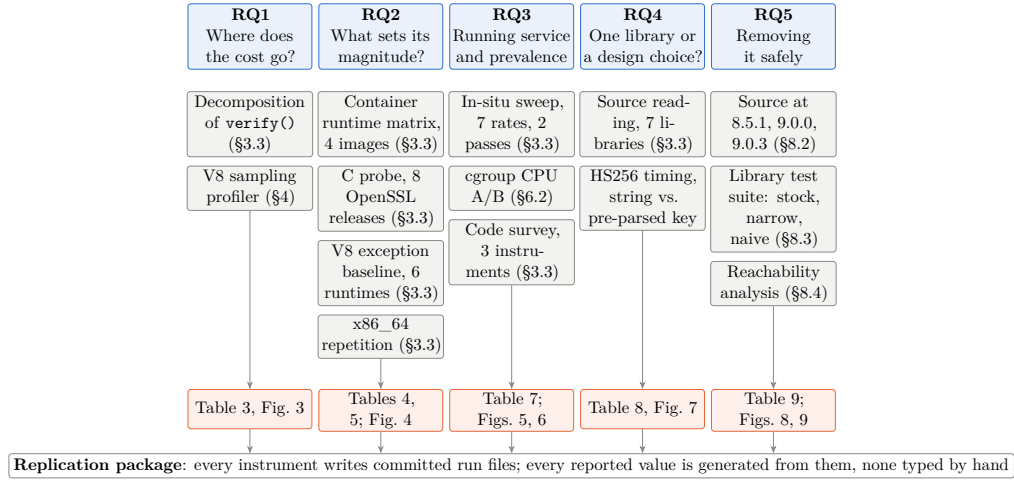


Figure 1: Study workflow. Top: the five research questions. Middle: the instruments that answer each, with the section describing it. Bottom: the tables and figures each produces. The C probe and the exception baseline (RQ2) also decompose the discarded parse for RQ1 (Table 4), and the call-site survey (RQ3) supplies the algorithm-pinning figure used in RQ5. All measurements feed one replication package, from which every reported value is generated.

174 tokens with a shared secret supplied as a string. Load was generated in open
 175 loop by k6 v2.2.0 on the same machine, so every in-situ window records the
 176 host load average observed during it. This is a controlled single-node testbed,
 177 not a production deployment; we call it the *containerised service*.

178 Table 2 lists every machine that produced a reported value. Absolute
 179 timings are comparable only within one machine; across machines we compare
 180 orderings and ratios. The x86_64 runs used two different cloud VMs: VM A,
 181 for the runs of the first revision, was no longer available when the second set
 182 of runs was made on VM B, so those could not be repeated on VM A.

183 3.2. Measurement protocol

184 The unit of measurement is one operating-system process, not one call.
 185 Each process performs a fixed number of timed iterations of a single condition
 186 after a warmup of half that number and contributes its median; reported
 187 figures are medians of per-process medians, with 95% percentile bootstrap
 188 intervals over processes (10,000 resamples) (Georges et al., 2007). With 15
 189 processes per condition these intervals are indicative rather than calibrated;
 190 the per-process values are in the replication package, and its analysis script

Table 2: Machines. CPU model strings are as reported by the operating system, nominal clock included; the Mac’s model and core count were not recorded by the run metadata. Every run’s own metadata file records its machine.

Machine	CPU	CPUs	OS	Produced
Mac host (arm64)	Apple silicon (not recorded)	–	macOS 26.5.2	Table 3, Fig. 3, profiler shares (§4.2), Fig. 9 (arm64)
Docker Desktop VM on the Mac (arm64)	as host	10 allotted	Linux; Kubernetes v1.32.2	Table 5, Fig. 5, Table 7
VM A (x86_64 cloud)	Intel Xeon Processor @ 2.80GHz	4	Linux 6.18.44-fc- v37, glibc 2.39	Table 4, Fig. 4, Table 8, Fig. 7, §5.2, Fig. 9 (VM A)
VM B (x86_64 cloud)	Intel Xeon Processor @ 2.10GHz	4	Ubuntu 24.04.4 LTS, Linux 6.18.44-fc- v50	Table 6, Table 10, §8.3 suite and equivalence runs, Fig. 9 (VM B), §7.1

191 can also report BCa intervals. Conditions are interleaved within each round,
192 and differences between conditions are reported as the median of per-round
193 *paired* differences. The largest between-process coefficient of variation is 3.5%
194 on the host.

195 3.3. Instruments

196 *Decomposing the call.* `jwt.verify()` of `jsonwebtoken` version 9.0.3 is de-
197 composed into the operations it performs, each timed in isolation: structural
198 decode, the HMAC primitive with a string and with a pre-parsed key, the
199 symmetric key constructor, and the asymmetric key parse the library attempts
200 first (Figure 2), both when it throws and when it succeeds. Each condition
201 runs in 15 processes of 40000 calls.

202 *Runtime matrix.* Containerisation is held constant and the runtime varied:
203 four container images spanning two OpenSSL series and three V8 major
204 versions, on one machine, at 10 processes of 20000 calls. The runtime identity
205 of each image is recorded with every row and cross-checked (0 mismatches).

```

120 if (secretOrPublicKey != null && !(secretOrPublicKey instanceof
    KeyObject)) {
121   try {
122     secretOrPublicKey = createPublicKey(secretOrPublicKey);
123   } catch (_) {
124     try {
125       secretOrPublicKey = createSecretKey(typeof
secretOrPublicKey === 'string' ? Buffer.from(
secretOrPublicKey) : secretOrPublicKey);
126     } catch (_) {
127       return done(new JsonWebTokenError('secretOrPublicKey is not
valid key material'))
128     }
129   }
130 }

```

Figure 2: Key resolution in `jsonwebtoken` version 9.0.3, `verify.js`, lines 120–130. Verbatim but for the common leading indentation, which is removed to fit the column; line numbers are the file’s own. A secret that is not already a `KeyObject` is offered to `createPublicKey()` first, and `createSecretKey()` is reached only from the exception handler. For an HMAC secret supplied as a string the outer `try` can never succeed, so the asymmetric parse is attempted and discarded on every validation.

206 *C probe.* To remove the JavaScript engine, we reproduced in C, call for call
207 from the Node source, what `createPublicKey()` asks of OpenSSL for an
208 HMAC secret (three PEM scans, a private-key parse and the error-queue
209 drain), compiled with one compiler (GCC 13.3.0) against 8 OpenSSL releases
210 from 3.0.16 to 3.6.1 built identically from source, at 15 processes per release.

211 *Exception baseline.* An `Error` of the same message, properties and code as
212 Node’s (captured from a real failure) is thrown from 8 frames and discarded,
213 with no OpenSSL call, on six official Node runtimes. With the C probe this
214 gives a three-way decomposition of the discarded parse: the OpenSSL parse,
215 the V8 exception, and a remainder attributed to the binding between them.

216 *Other libraries.* We read the key-handling code of six further libraries,
217 the most used in their ecosystems (`fast-jwt`, `jose`, `PyJWT`, `golang-jwt`,
218 `nimbus-jose-jwt`, `jjwt`), and timed HS256 verification in each with the
219 secret as a string (or the rawest accepted form) and as the library’s own
220 pre-parsed key object, at 15 processes of 20000 calls.

221 *Second architecture.* The host decomposition, the runtime matrix, the C
222 probe, the exception baseline and the cross-library comparison were measured
223 on an x86_64 virtual machine (Intel(R) Xeon(R) Processor @ 2.80GHz, four
224 vCPUs) on shared cloud infrastructure. Following Laaber et al. (2019), we
225 take only orderings and ratios between interleaved conditions from it. Two
226 Node 22 builds appear in these runs: the repetition of the host decomposition
227 used that machine’s own Node 22.22.2 (OpenSSL 3.5.5), while the exception
228 baseline used the official Node 22.23.3 release binary, which bundles OpenSSL
229 3.5.8.

230 *In-situ measurement.* The containerised service exposes a histogram timing
231 the verification call alone, read before and after each of seven arrival-rate
232 steps; means are exact differences of `_sum` over `_count`. The sweep ran twice
233 on one process, ascending then descending. An A/B at one rate, validation
234 enabled and then disabled, reads the processor time charged to the container
235 from cgroup accounting.

236 *Prevalence.* Three instruments with different frames, captured on 2026-09-18:
237 code-search co-occurrence counts, a classified convenience sample of call sites
238 (one file per repository, key argument traced within its file), and a targeted
239 counter-search that fetches every file co-occurring the library with the pre-
240 parsing interface and adjudicates it by hand. None supports a population
241 proportion, so results are reported *in our sample*.

242 *Distribution-packaged Node.js.* To test whether the OpenSSL 3.0 case survives
243 on supported systems, an Ubuntu 24.04 image with the distribution’s own
244 `nodejs` package, which links the system OpenSSL, is measured beside two
245 official images under the runtime-matrix protocol (Section 5.3).

246 *Rejected tokens.* A further harness times `verify()` on well-formed tokens that
247 the client can build without the secret, one declaring RS256 and one declaring
248 HS256 with a signature not made with the secret, under the stock library,
249 both fixes of Section 8.3 and a pre-parsed key; a second script compares the
250 stock library and each fix on every combination of key material, token and
251 algorithm option; and the library’s own test suite is run against each variant
252 on every runtime (Section 8.4).

253 *Library survey.* To move RQ4 from existence towards prevalence, a frame
254 of npm packages is drawn from the registry’s search interface with stated

255 queries and a weekly-download threshold, and a detector built on a JavaScript
 256 parser flags every `try/catch` whose `try` calls a key or certificate parser and
 257 whose handler falls back to a different key constructor. Every hit is read and
 258 adjudicated by hand (Section 7.1).

259 4. RQ1: Where the cost goes

Table 3: Cost of each operation in the validation path. Medians of per-process medians over 15 processes of 40000 calls; 95% percentile bootstrap intervals over processes, which with 15 units are indicative rather than calibrated (Section 3.2).

Operation	Median (μ s)	95% interval (μ s)
<code>verify()</code> , string secret	29.92	[29.75, 30.12]
discarded asym. parse	18.04	[17.83, 18.25]
successful asym. parse	13.83	[13.62, 13.92]
<code>verify()</code> , pre-parsed key	6.042	[5.958, 6.125]
HMAC, string key	2.208	[2.208, 2.250]
HMAC, pre-parsed key	2.166	[2.125, 2.167]
structural decode	1.292	[1.250, 1.292]
symmetric key constructor	0.583	[0.542, 0.583]
timer overhead	0.042	[0.042, 0.042]

260 Table 3 and Figure 3 give each operation in the validation path. The
 261 signature primitive costs 2.208 μ s of a 29.92 μ s call. The conversion to which
 262 the remainder is most readily attributed, the symmetric key constructor, costs
 263 0.583 μ s. The discarded parse that precedes it costs 18.04 μ s, 60% of the call
 264 and its largest single component. Supplying a pre-parsed key instead of a
 265 string reduces the call by 23.92 μ s [23.58, 24.00]. The asymmetry often read
 266 as conversion being dearer than parsing is backwards: a *successful* asymmetric
 267 parse costs 13.83 μ s, so the RSA path’s parse is useful work while the HMAC
 268 path’s is waste.

269 4.1. What the discarded parse is made of

270 Table 4 decomposes the discarded parse on six official runtimes. On every
 271 runtime bundling OpenSSL 3.0 the C parse alone matches the probe to within
 272 a few percent (101% of it under Node 18.20.8): the discarded parse *is* the

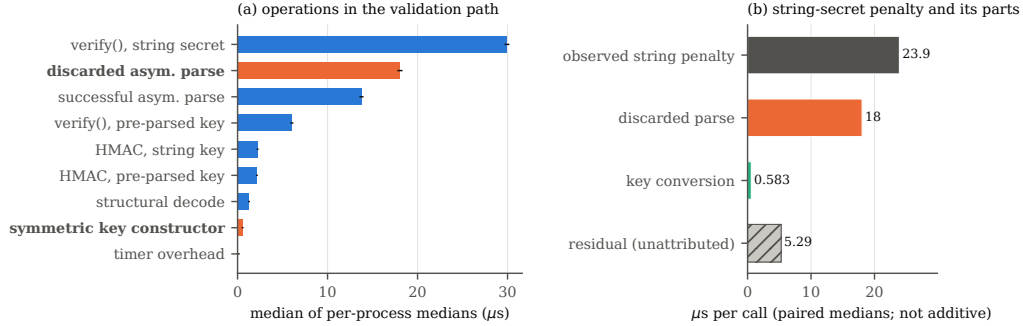


Figure 3: (a) Every operation in Table 3, on a linear scale, with the two components the paper turns on emphasised; whiskers are the 95% percentile bootstrap intervals over 15 processes. (b) The 23.92 μs penalty for supplying a string secret rather than a pre-parsed key, beside the discarded parse, the key conversion and the unattributed residual. Each bar is a median of per-round paired differences or of per-process medians, not part of one measurement, so the bars are not additive; the 5.292 μs residual is the part of the penalty the two measured components leave unexplained (Section 4.2).

Table 4: The discarded parse decomposed, per runtime, in μs (x86_64). *Probe*: `createPublicKey(secret)` inside Node, thrown and caught. *C parse*: the same OpenSSL calls issued from C against the OpenSSL release that runtime bundles, built from source. *V8 exception*: an `Error` of the same shape, message and properties thrown from 8 frames and discarded, with no OpenSSL call. *Binding*: the probe less the other two; its interval, from independent resampling of the three runs, is in the replication package. Every column is a median of per-process medians over 15 processes, on VM A (Table 2); absolute values are not comparable with other machines. The C column uses a stock build of each release, not Node’s own build of it, so the binding column is approximate: it is negative on two OpenSSL 3.0 rows, and the three-way split is interpretable only for the OpenSSL 3.5 rows (Section 4.1).

Runtime	OpenSSL	Probe	C parse	V8 exception	Binding
Node 18.20.8	3.0.16	390.5	394.4	4.781	-8.694
Node 20.20.2	3.0.19	443.4	431.3	4.796	7.292
Node 23.11.1	3.0.16	383.5	394.4	3.959	-14.88
Node 22.23.3	3.5.8	18.98	8.447	4.021	6.515
Node 24.21.0	3.5.8	19.54	8.447	4.136	6.952
Node 26.10.0	3.5.8	18.01	8.447	3.421	6.145

273 OpenSSL parse and would cost the same with no JavaScript engine. The V8
274 exception is small and nearly constant, about 4.781 μs (1.2% of the probe

under Node 18.20.8), half of it stack capture. Where OpenSSL 3.5 makes the parse cheap, the three parts are comparable: under Node 26.10.0 the parse is 47%, the exception 19.0% and the binding 34.1%. The three-way split is interpretable only for these OpenSSL 3.5 rows. On the OpenSSL 3.0 rows the binding comes out negative ($-8.694 \mu s$ under Node 18.20.8), because our stock C build of each release and Node’s own build of it differ by more than the binding costs; there only the dominance of the parse is meaningful. Either way the component is a discarded *parse*, not an exception cost.

4.2. Checks on the decomposition

The discarded parse and the conversion account for 77.9% of the string penalty. The $5.292 \mu s$ residual $[4.916, 5.501]$ presumably lies in the library’s own type and claim checking, but it was not measured directly and is reported as unattributed. Because the decomposition relies on differencing, it was checked against V8’s sampling profiler: the asymmetric key parse accounts for a median 66.2% of self time on the host (95.3% under Node 18 and 61.5% under Node 26), and does not appear among the top frames of the pre-parsed path. The two methods agree on which component dominates, not on its exact share: differencing gives 60% and the profiler 66.2% on the host. They measure different quantities. The differenced share divides the median of the parse, timed alone in its own processes, by the median of the whole call; the profiler’s share attributes sampled time inside one validation loop to the innermost frame, so work the parse frame triggers (including the construction of its exception) is counted with it, and sampling carries attribution error of its own (Mytkowicz et al., 2010). We therefore rely on the ordering, not on either percentage.

5. RQ2: What governs the magnitude

5.1. What the runtime matrix shows

Table 5 varies the runtime with containerisation held constant. The discarded parse spans $25.36\times$ across the four images, while pre-parsed validation spans only $1.26\times$. Node 18 and Node 20 ship different V8 major versions and share the OpenSSL 3.0 series; both pay of order $373.79\text{--}420.62 \mu s$, so within that series the engine does not move the cost. A successful parse moves with the failing one ($11.74\times$), so the effect concerns key parsing in general. The matrix alone cannot separate OpenSSL from the engine across the series

Table 5: The same measurements across four container runtimes on one machine (the Docker Desktop VM of Table 2, arm64; absolute values are not comparable with other machines), 10 processes of 20000 calls each, in μ s. *Discarded parse*: `createPublicKey()` on the HMAC secret, which fails. *Successful parse*: `createPublicKey()` on an RSA-2048 public key. *Pre-parsed*: `verify()` with a pre-parsed key. Across the four images the discarded parse spans $25.36\times$, the successful parse $11.74\times$ and pre-parsed validation $1.26\times$.[‡]From Table 3; not containerised. [†]The Homebrew OpenSSL the host’s Node links, inferred from the run record rather than recorded per row (Section 3.1).

Runtime	OpenSSL	V8	Discarded parse	Successful parse	Pre-parsed <code>verify()</code>
Node 18 musl	3.0.16	10	373.79	115.83	7.58
Node 20 musl	3.0.19	11	420.62	142.38	7.25
Node 26 musl	3.5.7	14	22.94	15.54	6.56
Node 26 glibc	3.5.7	14	16.58	12.12	6.04
host [‡]	3.6.3 [†]	14	18.04	13.83	6.042

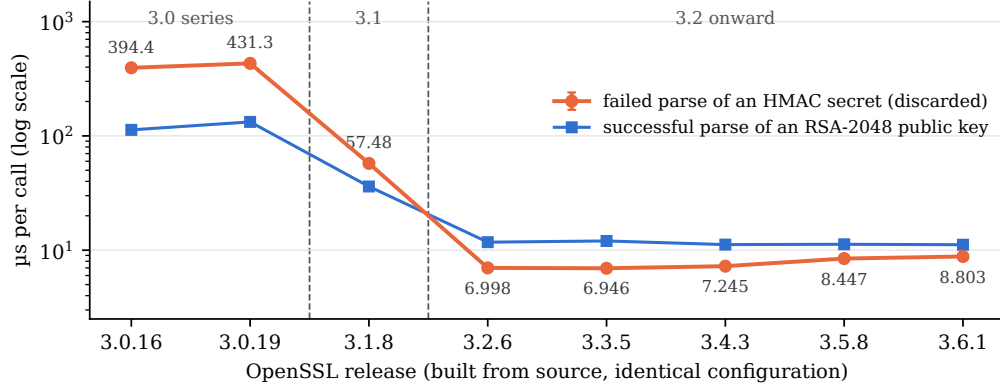


Figure 4: The failed parse of an HMAC secret issued from C, with no JavaScript engine, against 8 OpenSSL releases built from source with one configuration, beside a successful parse of an RSA-2048 public key through the same path. Whiskers are 95% percentile bootstrap intervals over 15 processes and are narrower than the markers. The cost falls in two steps, at 3.1 and 3.2, and is flat thereafter.

309 boundary, since the fast images also carry a newer engine and a different
310 build; the crossed designs below do.

311 *5.2. Caused by OpenSSL*

312 In plain C, with nothing else varying, the failure costs 394.4 μ s on OpenSSL
313 3.0.16 and 8.447 μ s on 3.5.8, a ratio of 46.69 [46.16, 46.90] (Figure 4). The
314 magnitude is therefore *caused* by the OpenSSL release, for this code path.
315 Almost all of it is the private-key parse (99% on 3.0.16), where OpenSSL 3
316 hands the input to its provider-based decoders. The cost fell in two steps, to
317 57.48 μ s in 3.1.8 and to 6.998 μ s in 3.2.6, and is flat thereafter. The engine
318 is excluded from the other direction too: Node 23.11.1 pairs a *newer* V8
319 with the *older* OpenSSL 3.0.16 and pays 383.5 μ s, against 18.98 μ s for Node
320 22.23.3 with OpenSSL 3.5.8.

321 *A second architecture.* On x86_64 every OpenSSL 3.0 image is again slower
322 than every OpenSSL 3.5 image (the split holds): the discarded parse spans
323 30.50 $\times$ across the images while pre-parsed validation spans 1.31 $\times$. The host
324 decomposition keeps its ordering: the discarded parse is 19.42 μ s of a 36.48 μ s
325 call (53%).

326 *5.3. Distribution-packaged Node.js*

327 The OpenSSL 3.0 series is not confined to end-of-life Node.js images.
328 Ubuntu 24.04 LTS packages Node.js 18.19.1, built against the system’s shared
329 OpenSSL rather than a bundled copy; in its current image that is OpenSSL
330 3.0.13. The package (18.19.1+dfsg-6ubuntu5) is in the *universe* component,
331 which Ubuntu describes as “community and extra packages” whose security
332 coverage is provided by Expanded Security Maintenance through Ubuntu Pro,
333 and the archive held no newer version on 2026-09-30.¹ It is nonetheless what
334 `apt install nodejs` installs on a current LTS system. Debian 12’s packaged
335 Node.js could not be measured: the Debian archive was not reachable from
336 VM B, and the run is scripted in the replication package. Table 6 measures
337 it beside two of the official images of Table 5, on one x86_64 virtual machine
338 (4 vCPUs) in one run (2026-09-30), with the protocol of Section 3.2.

339 On Ubuntu 24.04’s own Node.js the discarded parse costs 424 μ s, the same
340 order as on the end-of-life `node:18` image measured alongside it (636 μ s) and
341 an order of magnitude above the OpenSSL 3.5 image (39.6 μ s). Supplying the
342 secret as a string costs 442 μ s per validation [440, 443] against a pre-parsed
343 key; the narrow fix brings the call to 12.5 μ s and the key-material fix of

¹<https://ubuntu.com/about/release-cycle>; the package record and the page excerpt are in the replication package.

Table 6: Distribution-packaged Node.js against official images, all on VM B (Table 2) in one run; absolute values are not comparable with other machines, including the arm64 values of Table 5. 10 processes of 20,000 calls per condition (0 invocations failed), in μs (medians of per-process medians). `verify()` with a string secret under the stock library, the narrow fix and the key-material fix of Section 8.3, and with a pre-parsed key. *Penalty*: median of per-round paired differences between the stock string and pre-parsed calls, with its 95% bootstrap interval.

Runtime	OpenSSL	createPublicKey()		verify()				Penalty	95% interval
		Discarded	Successful	Stock	Narrow	Key-mat.	Pre-parsed		
Ubuntu 24.04, Node 18.19.1	3.0.13 (system)	424	124	451	12.5	12.7	9.87	442	[440, 443]
node:18.20.8-alpine	3.0.16 (bundled)	636	193	689	12.8	12.8	10.4	678	[677, 686]
node:26.6.0-alpine	3.5.7 (bundled)	39.6	19.0	58.9	9.65	9.72	8.30	50.5	[50.1, 51.5]

344 Section 8.3 to 12.7 μs . The OpenSSL 3.0 price is therefore not an artefact of
345 end-of-life images: a service that runs the Node.js its LTS distribution ships
346 pays it today. How many deployments do so we cannot say; we have no data
347 on the runtimes `jsonwebtoken` users deploy.

348 6. RQ3: The containerised service, and prevalence

349 6.1. In-situ cost

350 Figure 5 shows the in-situ cost at seven arrival rates. The cost is present
351 in the containerised service and far larger than the host figure, consistent with
352 Section 5: the service’s runtime carries the older OpenSSL, whose discarded
353 parse costs 373.79 μs , giving an expected total near 381 μs . Wall-clock means
354 of 453–1028 μs exceed that; contention inflates durations, so they are upper
355 bounds. Two features of Figure 5 are not explained by validation and we
356 draw no conclusion from them. The cost falls as the arrival rate rises, and
357 the descending pass differs from the ascending one systematically rather
358 than randomly. Plausible causes, none of which we isolated, are a fixed
359 per-window overhead amortised over more calls at higher rates; processor
360 frequency scaling and idle-state exit latency, which penalise sparse arrivals;
361 cache and branch-predictor residency, which improve with call density; and
362 continued JIT tier-up and garbage-collection timing, which differ between a
363 process’s first and second pass. The magnitude claim rests instead on the
364 processor-time measurement below.

365 6.2. Corroboration: the cost is processor time

366 To separate work from waiting, Table 7 compares the same service with
367 validation enabled and disabled, reading processor time from cgroup account-

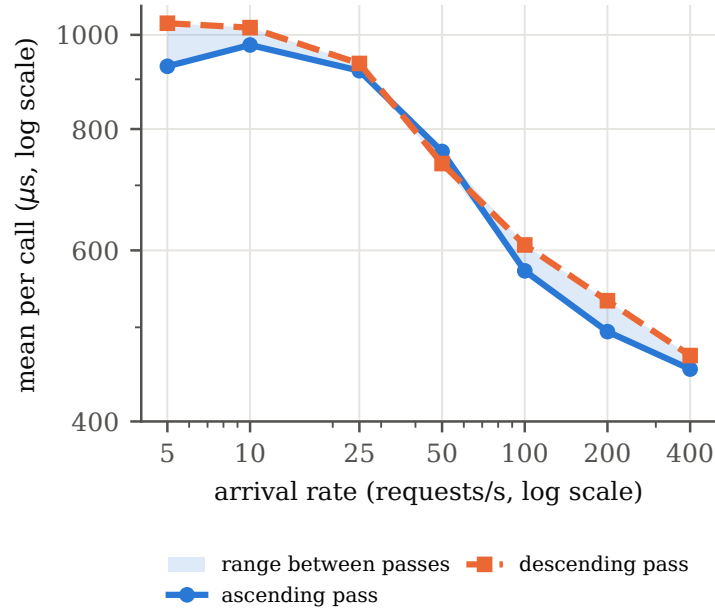


Figure 5: In-situ validation cost against arrival rate, string secret. The two passes are drawn separately rather than averaged, because they differ systematically and averaging would conceal that. The shaded band is the range between the passes, which is two observations rather than an interval.

Table 7: Validation enabled against validation disabled, at one arrival rate, two 180 s windows on the same pod and image, 36001 requests each. A signed token is on the wire in both arms; the disabled arm removes only the handler’s `jwt.verify()` call. Application and sidecar CPU are means of one-second cgroup samples in the window; CPU per call is that mean divided by the achieved rate. Event-loop lag is the mean of the per-scrape p99. Host load is a single sample at window start.

	disabled	enabled
Achieved rate (req/s)	200.006	200.006
Application CPU (millicores)	69.45	146.38
CPU per call (μs)	347.3	731.9
Sidecar CPU (millicores)	53.40	49.13
Event-loop lag (ms)	3.137	3.159
Client mean latency (ms)	1.6802	2.0332
Host load, 1 min	3.53	5.11

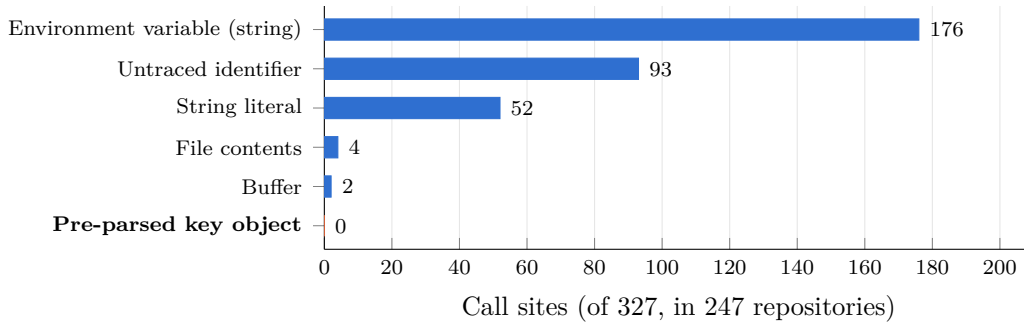


Figure 6: Where the secret passed to the validator comes from, for the 327 call sites classified in our sample. No call site passes a pre-parsed key object, the one form that avoids the discarded parse for every input. Independently, 309 of the 327 call sites (94%) pass no algorithm restriction. Counts describe the sample, not the population (Section 3.3).

368 ing rather than from a clock. Enabling validation raised the application’s
 369 processor time per request from 347.3 to 731.9 μ s: validation accounted for
 370 53% of the application’s CPU per request, a factor of 2.11 over the service
 371 without it. The added 384.6 μ s is of the same size as the 381 μ s the runtime
 372 matrix predicts for the same work on the same runtime, from instruments that
 373 share no code. A request that queues accrues wall-clock time but no CPU,
 374 so this figure excludes queueing delay; it can still be inflated by contention
 375 for caches, SMT siblings and processor frequency on the shared host. The
 376 sidecar’s CPU did not rise with the application’s, and the one-second CPU
 377 samples of the two arms do not overlap (disabled never exceeded 100.577
 378 millicores, enabled never fell below 114.963). This is one pair of consecutive
 379 windows at different host load, so the difference carries no interval; a repeated,
 380 alternating design is prepared for the authors’ testbed (Section 10).

381 6.3. How common is the avoiding pattern, in our sample?

382 In code-search counts, no cell exceeds 0.0303% of files importing the library
 383 that also mention the pre-parsing interface. Of 327 call sites classified across
 384 247 repositories, 0 pass a pre-parsed key (Figure 6), and 309 (94%) pass no
 385 algorithm restriction. Call sites are found by resolving the local binding of the
 386 import, which adds 12 that the conventional `jwt.verify()` spelling misses.
 387 The targeted counter-search fetched 191 files, 120 with a validation call in
 388 112 repositories. Of the 13 call sites it flagged automatically, hand inspection
 389 found 5 false positives and 1 that could not be resolved within its file; 7 were

Table 8: HS256 verification across seven JWT libraries in four languages (x86_64), in μ s. *Asym. first?*: given a string HMAC secret, does the library attempt to parse it as an asymmetric key, as read from its source. *String* and *pre-parsed*: the secret in the form the library’s documentation shows, and in the library’s own key object built once (Section 3.3). *Penalty*: median of per-round paired differences, with a 95% bootstrap interval over rounds. Medians of per-process medians over 15 processes of 20000 calls. Absolute costs measure different amounts of work in different runtimes and are not a ranking of libraries. ^aNo string accepted for an HMAC key; the rawest accepted form (bytes) is used. ^bA string is accepted only by a deprecated setter that resolves it once, when the parser is built.

Library	Lang.	Version	Crypto backend	Asym. first?	String	Pre-parsed	Penalty	95% interval
jsonwebtoken	JS	9.0.3	OpenSSL 3.0.19	yes	476.2	10.23	465.9	[465.2, 467.8]
jsonwebtoken	JS	9.0.3	OpenSSL 3.5.8	yes	34.22	9.844	24.56	[24.07, 24.82]
fast-jwt	JS	6.3.3	OpenSSL 3.0.19	no	7.564	7.570	0.070	[-0.116, 0.111]
fast-jwt	JS	6.3.3	OpenSSL 3.5.8	no	7.917	7.934	-0.068	[-0.134, 0.197]
jose	JS	6.2.12	OpenSSL 3.0.19	no ^a	121.2	78.42	42.66	[38.50, 45.89]
jose	JS	6.2.12	OpenSSL 3.5.8	no ^a	98.89	73.98	24.80	[21.91, 26.51]
PyJWT	Python	2.15.0	hashlib, OpenSSL 3.0.13	no	41.33	41.28	-0.103	[-0.457, 0.338]
golang-jwt	Go	5.3.1	Go crypto/hmac	no ^a	4.810	4.764	0.034	[-0.042, 0.117]
nimbus-jose-jwt	Java	10.10	JCA SunJCE	no	4.358	4.229	0.304	[-0.098, 1.016]
jjwt	Java	0.13.0	JCA SunJCE	no ^b	12.15	12.33	-0.406	[-0.715, 0.648]

genuine, in 6 projects. Even among repositories that know the interface exists, only 5.36% use it: the pattern is rare, not absent. The zero in the classified sample may be biased downwards: keys are traced within one file only, and 93 of the 327 call sites pass an identifier whose origin lies in another file and could, in principle, be a key object built there. No file in the sample mentions the pre-parsing interface in any spelling, which weighs against that, but it is not proof.

7. RQ4: One library, or a design choice?

A cost found in one library becomes a statement about a class of defect only if its mechanism can be named independently and predicts which other libraries have it. The mechanism here is *resolving key material of unknown type by attempting the asymmetric parse first, and treating its failure as the signal that the material is a shared secret*. Table 8 and Figure 7 test that prediction.

Only `jsonwebtoken` attempts the asymmetric parse. The others avoid it in one of three ways: *inspect, then dispatch* (`fast-jwt` and `PyJWT` examine the key text and call exactly one constructor); *refuse the ambiguous input* (`jose` and `golang-jwt` do not accept a string HMAC key); or *type the key* (`jjwt` requires a `SecretKey`; `nimbus-jose-jwt` converts a string’s bytes without

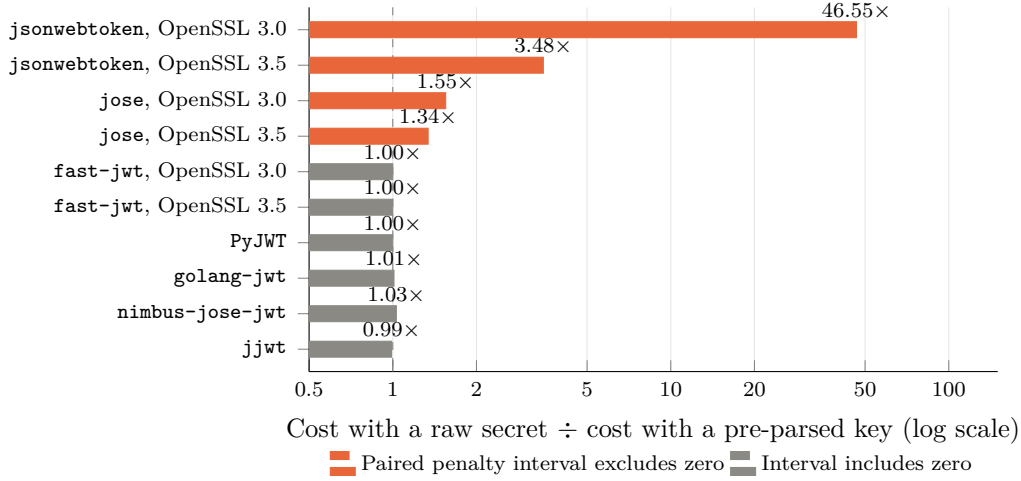


Figure 7: How much each library pays for being handed its raw secret rather than its own pre-parsed key object: the ratio of the two costs in Table 8, on a log scale. A ratio of one (dashed line) means no per-call penalty. Orange: the paired 95% interval on the penalty excludes zero. `jsonwebtoken` and `jose` both pay for per-call key handling; under OpenSSL 3.5.8 their absolute penalties are indistinguishable, and `jsonwebtoken`’s larger ratio there reflects `jose`’s higher baseline. Only `jsonwebtoken` performs a failing asymmetric parse, and only its penalty changes by an order of magnitude with the OpenSSL release.

409 parsing them).

410 Of the seven libraries, two pay for a raw secret, in the sense that the paired

411 interval on the penalty excludes zero; the other five pay nothing measurable.

412 The two are `jsonwebtoken` and `jose`, and they pay for different reasons. `jose`

413 parses nothing that can fail, but imports raw bytes into a WebCrypto key on

414 every call. Under OpenSSL 3.5.8 the two absolute penalties are statistically

415 indistinguishable: $24.56 \mu s$ [24.07, 24.82] for `jsonwebtoken` and $24.80 \mu s$

416 [21.91, 26.51] for `jose`. `jsonwebtoken`’s larger ratio in Figure 7 there reflects

417 `jose`’s higher baseline, not a larger penalty. The libraries separate under

418 OpenSSL 3.0.19: `jsonwebtoken`’s penalty rises to $465.9 \mu s$ and `jose`’s to

419 $42.66 \mu s$, about 11 times less. The distinguishing property of `jsonwebtoken`

420 is therefore not that it alone pays for per-call key handling, but that it does

421 so through a *failing* parse, whose price follows the OpenSSL release by an

422 order of magnitude. All seven libraries validate the same kind of token with

423 the same primitive, so the cost follows a design choice, resolving key material

424 on every call, and not JWT validation as such. The failing parse is the form
425 of that choice most exposed to the cryptographic library underneath, and
426 the class of order-of-evaluation defect the profiler of Selakovic et al. (2017)
427 targets.

428 *7.1. How often is a failing parse used as a type test?*

429 Seven libraries show that the design choice exists; they do not show how
430 often it is made. We therefore searched for it systematically in one ecosystem
431 (Section 3.3). The first frame holds the npm packages returned for six JWT
432 and JOSE keyword queries with at least 50,000 weekly downloads (64 of
433 1727 returned, captured 2026-09-30); the second, wider frame holds packages
434 returned for nine cryptography, signature, certificate and authentication
435 queries with at least 500,000 weekly downloads (164 of 6222). The detector
436 parsed all 11767 and 9989 JavaScript files of the published tarballs without
437 parse failures in either frame.

438 It flagged 1 package in the first frame and 0 in the second. The one package
439 is `jsonwebtoken` itself, at 2 sites, both confirmed by reading and both on a
440 per-call path: the `verify()` block studied here, and the same shape in `sign()`,
441 which offers the configured secret to `createPrivateKey()` before falling back
442 to `createSecretKey()`. Signing therefore pays a discarded parse too. Timed
443 on the runtimes of Table 6, `sign()` with a string secret costs 443 μ s on Ubuntu
444 24.04’s Node.js, against 7.37 μ s with a pre-parsed key (a paired penalty of
445 436 μ s [434, 440]); 652 against 7.95 μ s on `node:18.20.8-alpine`; and 51.1
446 against 6.55 μ s on `node:26.6.0-alpine`. An issuer that signs with a string
447 secret on OpenSSL 3.0 pays the same order of cost as a verifier. Within these
448 frames, then, the failing parse used as a type test is not a common idiom
449 but a property of one library, whose reach comes from that library’s own
450 adoption. The detector matches one syntactic shape, so a type test written
451 differently (for example by inspecting an error code, or through a third-party
452 parser) is not flagged, and the count is a lower bound within the frames; other
453 ecosystems remain to be surveyed.

454 **8. RQ5: Removing the cost safely**

455 *8.1. Threat model, and the discarded parse precedes authentication*

456 The adversary is a remote client that can place arbitrary bytes where the
457 service expects a bearer token, including every header field, but does not
458 know the shared secret. The asset is key-type separation: a token declaring

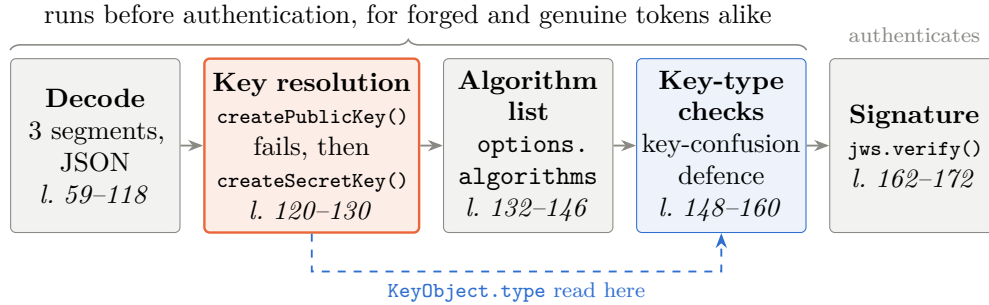


Figure 8: Order of operations in `verify()` at tag v9.0.3, with the file’s line numbers. Every step up to and including the algorithm and key-type checks runs before the signature is verified, and so runs for a forged token exactly as for a genuine one. The discarded parse sits inside key resolution; the key-type checks that implement the library’s key-confusion defence read the type of the key object resolution produces (dashed arrow). Pinning permitted algorithms is enforced at the third step and therefore does not prevent the second.

459 an HMAC algorithm must never be verified against asymmetric key material,
 460 nor the reverse (Sheffer et al., 2020). We measure what a rejected forged
 461 token costs per call (Section 8.4), but we drove no adversarial traffic at a
 462 service and make no claim about availability or capacity. Figure 8 shows
 463 the order of checks in `verify()`. Reaching key resolution requires only a
 464 well-formed token; the permitted-algorithm list, the key-type checks and the
 465 signature all come later. A token that will be rejected therefore pays the
 466 discarded parse in full, and restricting permitted algorithms does not avoid it.

467 8.2. The discarded parse arrived with a key-confusion fix

468 Comparing `verify.js` in versions 8.5.1, 9.0.0 and 9.0.3: in 8.5.1 the
 469 secret is passed to the signature routine as given, and the resolution block of
 470 Figure 2 first appears in 9.0.0, the release that fixed CVE-2022-23541 and
 471 CVE-2022-23539 (Auth0, 2022). The checks implementing those fixes (lines
 472 148–160) operate on the key object the block produces. The discarded parse
 473 is part of a security fix, not an oversight.

474 8.3. Two fixes, and a constraint on both

475 *The constraint.* The *success* of the parse on asymmetric key material produces
 476 the key object whose type the key-confusion check depends on. Removing
 477 the attempt for every string (the obvious optimisation) makes a PEM public
 478 key configured where a secret is expected resolve as a secret, and produces

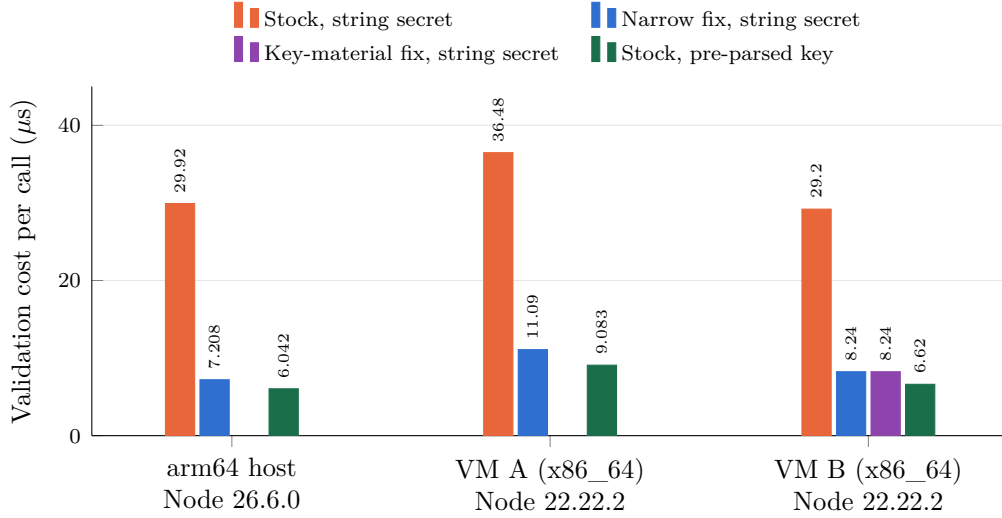


Figure 9: Per-call HS256 validation cost with a string secret under the stock library, the narrow fix and the key-material fix, and with a pre-parsed key under the stock library (medians of per-process medians). Each group is one machine (Table 2); absolute values are not comparable across groups. The key-material fix was measured only on VM B; its arm64 run is in the replication package’s list of pending experiments.

2 failures in the library’s key-confusion tests (“should not allow HMAC verification with an RSA key in PEM format” and, under “signing with pub key as symmetric”, “should not verify”). A fix must therefore skip the attempt only for key material that cannot be asymmetric. The existing suite already guards the property; what is missing is visibility of the coupling to someone changing the code.

The narrow fix. Our first fix dispatches on the token’s declared algorithm: when the token declares an HMAC algorithm *and* the secret is a string with neither a PEM header nor a JSON object, the secret is resolved directly. It takes the call from 29.92 μ s to 7.208 μ s on the arm64 host, a factor of 4.15, with no change in calling code, and leaves the library’s suite unchanged (511 passing, 0 failing).

The key-material fix. The declared algorithm is not needed. In Node.js a string passed to `createPublicKey()` is parsed as PEM, so a string with no PEM header can never produce a public key: for such a string the stock library always reaches `createSecretKey()` through the exception. Dispatching on

Table 9: Which tokens reach the discarded parse, for a service configured with an HMAC shared secret, as read from the source at tag v9.0.3 and the patches of Section 8.3, and confirmed by the measurements of Table 10. “Well-formed”: three segments, JSON header and payload, non-empty signature. No row requires knowledge of the secret.

Configuration	Tokens that reach the parse
Stock, string secret	every well-formed token, forged or genuine
Narrow fix, plain string secret (no PEM or JWK form)	every well-formed token whose header declares a non-HMAC algorithm; no such token can be valid for this service
Key-material fix, plain string secret	none
Pre-parsed <code>KeyObject</code>	none

495 the key material alone (a string with neither a PEM header nor a JSON
 496 object goes straight to `createSecretKey()`, whatever the token declares)
 497 therefore yields the same key object for every token, and keeps everything
 498 that might be asymmetric material on the original path. We tested this rather
 499 than relying on the argument. On each of 4 runtimes the library’s suite at
 500 tag v9.0.3 gives 511 passing and 0 failing under this fix, as under the stock
 501 library and the narrow fix, and 515 passing with our four additional key-path
 502 tests; the unrestricted optimisation gives 2 failures on every runtime. On the
 503 same machine (VM B, Node 22.22.2, OpenSSL 3.5.5) the key-material fix
 504 takes the call from 29.2 μ s to 8.24 μ s, against 8.24 μ s for the narrow fix and
 505 6.62 μ s with a pre-parsed key (Figure 9). What preserves the defence is the
 506 restriction on key material, not the declared algorithm; the naive optimisation
 507 fails because it drops that restriction.

508 8.4. What the fixes leave reachable

509 The narrow fix selects its fast path by the declared algorithm, which
 510 the token’s author chooses: a token declaring a non-HMAC algorithm falls
 511 through to the original resolution and pays the parse before being rejected
 512 (Table 9). It removes the cost from legitimate traffic, not from traffic an
 513 adversary constructs. The key-material fix and a pre-parsed key object remove
 514 it for every input.

515 *Measured.* Table 10 times the rejection of well-formed forged tokens,
 516 which a client can produce without the secret, under each configuration

Table 10: Cost of rejecting a forged token, in μs per `verify()` call (medians of per-process medians), for a service configured with an HMAC shared secret and `algorithms: ['HS256']`. *RS256 header*: a token declaring RS256 with a random signature, rejected by the algorithm list. *HS256 header*: a token declaring HS256 with a signature not made with the secret, rejected at signature verification. *Narrow* and *Key-mat.*: the narrow fix and the key-material fix of Section 8.3. All cells were measured on VM B (Table 2); absolute values are not comparable with those of other machines.

Runtime	OpenSSL	RS256 header				HS256 header			
		Stock	Narrow	Key-mat.	Pre-parsed	Stock	Narrow	Key-mat.	Pre-parsed
<code>node:18.20.8-alpine</code>	3.0.16	699	677	22.6	19.1	724	31.0	30.8	26.9
Ubuntu 24.04, Node 18.19.1	3.0.13	461	467	24.6	21.1	480	32.3	32.2	28.5
<code>node:26.6.0-alpine</code>	3.5.7	115	114	61.8	58.5	78.6	25.5	25.5	23.2
Node 22.22.2 (VM B)	3.5.5	35.5	35.9	16.7	14.5	41.1	21.7	21.5	19.3

(`bench/forged-token.js`; 10 processes of 10000 calls per cell, on VM B). The measurements confirm the source reading. A token declaring RS256 is rejected by the algorithm list, yet under the stock library and the narrow fix it first pays the discarded parse: on Ubuntu 24.04’s OpenSSL 3.0.13 461 μs and 467 μs , against 24.6 μs under the key-material fix and 21.1 μs with a pre-parsed key. A forged token declaring HS256 is cheap to reject under either fix. On `node:26.6.0-alpine` an RS256-header rejection costs more than an HS256 one even with a pre-parsed key. The difference is V8’s capture of the error’s stack trace at that throw site: run with stack capture disabled (`-stack-trace-limit=0`), it falls from 58.0 μs to 6.59 μs , while the HS256 rejection falls from 23.3 μs to 12.7 μs ; on the other three runtimes the RS256 rejection is the cheaper of the two with or without stack capture (10 processes per cell). These are per-call costs in a microbenchmark; we did not drive such tokens at a running service, and draw no conclusion about availability.

Behaviour under misconfiguration. Both fixes take their fast path only for a string that carries neither a PEM header nor a JSON object, so a PEM or JWK key mistakenly configured as the HMAC “secret” resolves exactly as it does today. We checked this rather than assuming it: for 13 kinds of configured key material (including PEM public and private keys, a PEM with leading whitespace, a PEM inside a Buffer, and a JWK string supplied as the secret), 364 combinations of key, token and algorithm option were verified through the stock library and each fix on each of 4 runtimes. 0 produced a different result or error under the narrow fix and 0 under the key-material fix (`bench/keypath-equivalence.js`).

```

const { createSecretKey } = require('node:crypto');
const jwt = require('jsonwebtoken');

// Once, at startup: build the key object.
const key = createSecretKey(Buffer.from(process.env.JWT_SECRET));

// Per request: pass the key object instead of the string.
jwt.verify(token, key, { algorithms: ['HS256'] });

```

Figure 10: The one-line change that removes the discarded parse for every input: the secret is converted to a `KeyObject` once, and `verify()` skips key resolution when handed one (Figure 2, line 120).

542 8.5. Reported upstream

543 An earlier issue² reports HS256 verification in 9.x 34 times slower than in
544 8.5.1 on OpenSSL 3.0, with no diagnosed cause. We reported the mechanism³,
545 and a pull request implementing the narrow fix was opened against it; the
546 key-material fix of Section 8.3 is simpler and closes the gap the narrow fix
547 leaves, and we recommend it upstream. At the time of this revision (30
548 September 2026) the pull request had not been merged, no release after 9.0.3
549 had been published, and the repository’s main branch had not changed since
550 June 2026.

551 9. Implications for practice

552 Table 11 maps each finding to its evidence and to the stakeholder who
553 can act on it.

554 *Application developers.* Pre-parse key material once, at startup (Figure 10).
555 It removes 23.92 μ s per validation on the host and far more on OpenSSL 3.0,
556 needs no architectural change and no change to the library, and is the only
557 remedy that removes the discarded parse for every input. Also restrict
558 permitted algorithms (Sheffer et al., 2020); that is an independent defence,
559 not a substitute.

²<https://github.com/auth0/node-jwt-token/issues/966>

³<https://github.com/auth0/node-jwt-token/issues/1046>

560 *Performance engineers.* Treat the base image as part of the measurement.
561 The same code differs by $25.36\times$ on this path across images because they
562 pin different OpenSSL releases, and a regression after a base-image bump
563 may have nothing to do with the application. Profile before relocating a cost
564 believed to be cryptographic to a sidecar or gateway.

565 *Library authors.* Do not use a failing parse as a type test. Inspect and
566 dispatch, refuse ambiguous input, or type the key, and resolve keys once
567 rather than on every call.

568 *Maintainers and reviewers.* Resolve a plain string secret directly, keeping
569 anything that might be asymmetric material on the original path: the key-
570 material fix of Section 8.3 is a change of a few lines. More generally, review
571 performance changes to verification code as security changes, run them against
572 the key-confusion tests, and time them on the oldest cryptographic library a
573 deployment may bundle.

574 10. Threats to validity

575 *Construct.* The decomposition relies on differencing, cross-checked by a pro-
576 filer whose share differs from the differenced one for the reasons given in
577 Section 4.2. The C probe reproduces Node’s OpenSSL calls but not Node’s
578 own build of OpenSSL, so the binding column of Table 4 is approximate;
579 the causal claim compares releases under one build and does not inherit this.
580 Every measurement uses HS256 with one secret of 34 characters and one token
581 of fixed size. The parse fails at the PEM-header check and in the private-key
582 decoder, and its cost does not depend on the secret’s length: a 16- and a
583 256-character secret cost $673\ \mu\text{s}$ and $679\ \mu\text{s}$ on OpenSSL 3.0.16 (Section 8.4).
584 A secret that begins with a PEM header takes the original path under both
585 fixes by design and was not timed. The security analysis is from source, the
586 library’s tests and the microbenchmarks of Section 8.4, not from adversarial
587 traffic against a service.

588 *Internal.* The in-situ evidence is one process sweeping seven rates twice, and
589 the A/B is one pair of consecutive windows at different host load with the
590 load generator on the same machine. CPU accounting excludes queueing
591 delay, but contention for caches, SMT siblings and processor frequency can
592 still raise the CPU time charged for the same work, so the A/B difference is an
593 upper bound on the work and carries no interval. An alternating design with

594 the load generator pinned or remote is prepared in the replication package
595 but was not run. The headline host figure predates the per-measurement
596 OpenSSL field; its version (3.6.3) is inferred from the run record, and a re-run
597 with the version recorded is prepared.

598 *External.* Primary measurements come from one ARM64 host; the x86_64
599 machines are shared virtual machines, used only for orderings and ratios.
600 The containerised service is a single-node testbed on an end-of-life Node 18
601 image; the OpenSSL 3.0 behaviour it shows is also present in Ubuntu 24.04's
602 own Node.js (Section 5.3), but the service itself was not re-measured on a
603 supported image. A service that caches verified tokens pays the cost once
604 per token. Seven libraries at one version each show a design choice; the npm
605 survey of Section 7.1 estimates how often a failing parse is used as a type test
606 within a stated frame of one ecosystem, not across ecosystems.

607 *Reliability of the survey.* Call-site buckets are assigned by rules that trace a
608 key within one file, and the counter-search and the library-survey hits were
609 adjudicated by one person. A blinded, stratified sheet of call sites, a codebook
610 and an agreement script for a second rater are in the replication package; the
611 second rating has not yet been done.

612 *Future work.* Driving forged tokens at the running service, repeating the A/B
613 as alternating windows on supported images, bisecting OpenSSL between 3.0
614 and 3.2, and extending the library survey to PyPI, Go and Maven would each
615 close a limitation stated above.

616 11. Conclusion

617 The per-request cost of token validation in `jsonwebtoken` is not the
618 signature ($2.208\ \mu\text{s}$ of $29.92\ \mu\text{s}$) and not key conversion ($0.583\ \mu\text{s}$). It is a
619 parse attempted in the wrong order and discarded on every request, 60% of
620 the call. Its price is set causally by the OpenSSL release beneath the runtime:
621 $394.4\ \mu\text{s}$ on 3.0.16 against $8.447\ \mu\text{s}$ on 3.5.8, and the slow case persists in the
622 Node.js that Ubuntu 24.04 ships. Per-call key handling is a design choice, not
623 a property of JWT; `jsonwebtoken` is distinguished by making it through a
624 failing parse, whose cost changes by an order of magnitude with the OpenSSL
625 release. That parse cannot be removed naively, because it is load-bearing
626 for a key-confusion defence, and it is the restriction on key material, not the
627 token's declared algorithm, that the defence needs: a fix that dispatches on

628 the key material alone passes the library’s suite, behaves identically to the
629 stock library in every case we tested, and removes the parse for every token.
630 Until such a fix ships, pre-parsing the key once achieves the same, yet no call
631 site in our sample does so.

632 The broader lesson is about attribution. The explanation that presented
633 itself before measurement was wrong twice, and the component that sets
634 the cost is one that measurement protocols rarely record. Performance
635 studies should record the identity of the components they measure, including
636 transitive dependencies such as the bundled cryptographic library, at the
637 granularity at which they can change.

638 **CRedit authorship contribution statement**

639 **Jannatul Ferdous Katha:** Conceptualization, Methodology, Software,
640 Investigation, Data curation, Formal analysis, Visualization, Writing – original
641 draft. **Tasmia Tahmid Prova:** Validation, Writing – review & editing. **Md**
642 **Kishor Morol:** Supervision, Writing – review & editing. **Tze Hui Liew:**
643 Supervision, Project administration, Writing – review & editing. **Dip Nandi:**
644 Supervision, Writing – review & editing.

645 **Declaration of competing interest**

646 The authors declare that they have no known competing financial interests
647 or personal relationships that could have appeared to influence the work
648 reported in this paper.

649 **Funding**

650 This research did not receive any specific grant from funding agencies in
651 the public, commercial, or not-for-profit sectors.

652 **Data availability**

653 The data supporting the results of this study are openly available in the
654 replication package (Katha et al., 2026), archived at Zenodo under [https:](https://doi.org/10.5281/zenodo.23011153)
655 [//doi.org/10.5281/zenodo.23011153](https://doi.org/10.5281/zenodo.23011153) and maintained in a public GitHub

656 repository.⁴ It contains all measurements, code and scripts that generate
657 every value and figure in this paper, and is released under the MIT licence.
658 The third-party survey corpus is not redistributed; a manifest pins each of its
659 502 files by repository, commit, path and content hash, and a script in the
660 package re-fetches every file from its repository and checks the hash, so the
661 corpus can be reconstructed wherever the files remain public. Experiments
662 that need the authors' testbed, and the second rating of the survey, are
663 scripted in the package with their exact commands.

664 **Ethics and responsible disclosure**

665 The study involves no human participants and no personal data, and
666 no attack was run against any system the authors do not control. The key-
667 confusion behaviour discussed is a publicly documented attack class, exercised
668 only through the library's own test suite.

669 **Declaration of generative AI and AI-assisted technologies in the** 670 **manuscript preparation process**

671 During the preparation of this manuscript, generative AI tools were used to
672 improve language and readability. The authors reviewed all content following
673 such use and take full responsibility for the contents of the manuscript.

674 **Acknowledgements**

675 The authors acknowledge ELITE Research Lab for providing the compu-
676 tational resources and support necessary for this research.

677 **References**

678 Auth0, 2022. CVE-2022-23539, CVE-2022-23541, CVE-2022-23540: Security update
679 for jsonwebtoken. Auth0 security bulletin, 21 December 2022, [https://auth0.](https://auth0.com/docs/secure/security-guidance/security-bulletins/2022-12-21-jsonwebtoken)
680 [com/docs/secure/security-guidance/security-bulletins/2022-12-21-j](https://auth0.com/docs/secure/security-guidance/security-bulletins/2022-12-21-jsonwebtoken)
681 [sonwebtoken](https://auth0.com/docs/secure/security-guidance/security-bulletins/2022-12-21-jsonwebtoken). Accessed 2026-09-28.

⁴<https://github.com/Katha-Sikdar/A-Pre-Authentication-Key-Parse-Dominates-JWT-Validation-in-jsonwebtoken-and-Is-Load-Bearing->

682 Barrett, E., Bolz-Tereick, C.F., Killick, R., Mount, S., Tratt, L., 2017. Virtual
683 machine warmup blows hot and cold. *Proceedings of the ACM on Programming*
684 *Languages* 1, 52:1–52:27. doi:10.1145/3133876.

685 Cabral, B., Marques, P., 2007. Exception handling: A field study in Java and
686 .NET, in: *ECOOP 2007 – Object-Oriented Programming*, Springer. pp. 151–175.
687 doi:10.1007/978-3-540-73589-2_8.

688 Chen, J., Shang, W., 2017. An exploratory study of performance regression
689 introducing code changes, in: *2017 IEEE International Conference on Software*
690 *Maintenance and Evolution (ICSME)*. doi:10.1109/ICSME.2017.13.

691 Cito, J., Schermann, G., Wittern, J.E., Leitner, P., Zumberi, S., Gall, H.C., 2017.
692 An empirical analysis of the Docker container ecosystem on GitHub, in: *2017*
693 *IEEE/ACM 14th International Conference on Mining Software Repositories*
694 *(MSR)*. doi:10.1109/MSR.2017.67.

695 Coarfa, C., Druschel, P., Wallach, D.S., 2006. Performance analysis of TLS web
696 servers. *ACM Transactions on Computer Systems* 24. doi:10.1145/1124153.11
697 24155.

698 Costa, D., Bezemer, C.P., Leitner, P., Andrzejak, A., 2021. What’s wrong with
699 my benchmark results? Studying bad practices in JMH benchmarks. *IEEE*
700 *Transactions on Software Engineering* 47, 1452–1467. doi:10.1109/TSE.2019.2
701 925345.

702 Davis, J.C., Williamson, E.R., Lee, D., 2018. A sense of time for JavaScript
703 and Node.js: First-class timeouts as a cure for event handler poisoning, in:
704 *Proceedings of the 27th USENIX Security Symposium*, pp. 343–359.

705 Detering, D., Somorovsky, J., Mainka, C., Mladenov, V., Schwenk, J., 2017. On
706 the (in-)security of JavaScript object signing and encryption, in: *Proceedings*
707 *of the 1st Reversing and Offensive-oriented Trends Symposium (ROOTS ’17)*.
708 doi:10.1145/3150376.3150379.

709 Egele, M., Brumley, D., Fratantonio, Y., Kruegel, C., 2013. An empirical study
710 of cryptographic misuse in Android applications, in: *Proceedings of the 2013*
711 *ACM SIGSAC Conference on Computer and Communications Security (CCS*
712 *’13)*. doi:10.1145/2508859.2516693.

713 Gaynor, A., Kehrer, P., 2026. The state of OpenSSL for pyca/cryptography.
714 pyca/cryptography project statement, <https://cryptography.io/en/latest/statements/state-of-openssl/>. Maintainer statement, not peer reviewed;
715 accessed 2026-09-24.

717 Georges, A., Buytaert, D., Eeckhout, L., 2007. Statistically rigorous Java perfor-
718 mance evaluation, in: *Proceedings of the 22nd Annual ACM SIGPLAN Confer-*
719 *ence on Object-Oriented Programming Systems, Languages and Applications*
720 *(OOPSLA ’07)*. doi:10.1145/1297027.1297033.

721 GitHub Advisory Database, 2015. Verification bypass in jsonwebtoken (CVE-2015-
722 9235, GHSA-c7hr-j4mj-j2w6). <https://github.com/advisories/GHSA-c7hr-j4mj-j2w6>. Accessed 2026-09-28.

723

724 Jin, G., Song, L., Shi, X., Scherpelz, J., Lu, S., 2012. Understanding and detecting
725 real-world performance bugs, in: Proceedings of the 33rd ACM SIGPLAN
726 Conference on Programming Language Design and Implementation (PLDI '12),
727 pp. 77–88. doi:10.1145/2254064.2254075.

728 Jones, M.B., Bradley, J., Sakimura, N., 2015. JSON Web Token (JWT). Request for
729 Comments RFC 7519. Internet Engineering Task Force. doi:10.17487/RFC7519.

730 Kalibera, T., Jones, R., 2013. Rigorous benchmarking in reasonable time, in:
731 Proceedings of the 2013 International Symposium on Memory Management
732 (ISMM '13). doi:10.1145/2464157.2464160.

733 Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D.M., Damian, D.,
734 2014. The promises and perils of mining GitHub, in: Proceedings of the 11th
735 Working Conference on Mining Software Repositories (MSR 2014). doi:10.114
736 5/2597073.2597074.

737 Kaltenecker, C., Mühlbauer, S., Grebhahn, A., Siegmund, N., Apel, S., 2023.
738 Performance evolution of configurable software systems: An empirical study.
739 Empirical Software Engineering 28. doi:10.1007/s10664-023-10338-3.

740 Katha, J.F., Prova, T.T., Morol, M.K., Liew, T.H., Nandi, D., 2026. Replication
741 package: a pre-authentication key parse dominates JWT validation in
742 jsonwebtoken. doi:10.5281/zenodo.23011153. Source repository: [https://github.com/Katha-Sikdar/A-Pre-Authentication-Key-Parse-Dominat](https://github.com/Katha-Sikdar/A-Pre-Authentication-Key-Parse-Dominates-JWT-Validation-in-jsonwebtoken-and-Is-Load-Bearing-)
743 [es-JWT-Validation-in-jsonwebtoken-and-Is-Load-Bearing-](https://github.com/Katha-Sikdar/A-Pre-Authentication-Key-Parse-Dominates-JWT-Validation-in-jsonwebtoken-and-Is-Load-Bearing-).
744

745 Kery, M.B., Le Goues, C., Myers, B.A., 2016. Examining programmer practices for
746 locally handling exceptions, in: Proceedings of the 13th International Conference
747 on Mining Software Repositories (MSR '16), pp. 484–487. doi:10.1145/290173
748 9.2903497.

749 Krüger, S., Späth, J., Ali, K., Bodden, E., Mezini, M., 2018. CrySL: An extensi-
750 ble approach to validating the correct usage of cryptographic APIs, in: 32nd
751 European Conference on Object-Oriented Programming (ECOOP 2018), pp.
752 10:1–10:27. doi:10.4230/LIPIcs.ECOOP.2018.10.

753 Laaber, C., Leitner, P., 2018. An evaluation of open-source software microbench-
754 mark suites for continuous performance assessment, in: Proceedings of the
755 15th International Conference on Mining Software Repositories (MSR '18).
756 doi:10.1145/3196398.3196407.

757 Laaber, C., Scheuner, J., Leitner, P., 2019. Software microbenchmarking in the
758 cloud. how bad is it really? Empirical Software Engineering 24. doi:10.1007/s1
759 0664-019-09681-1.

760 Lazar, D., Chen, H., Wang, X., Zeldovich, N., 2014. Why does cryptographic
761 software fail? A case study and open problems, in: Proceedings of the 5th Asia-
762 Pacific Workshop on Systems (APSys '14). doi:10.1145/2637166.2637237.

763 Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P.F., 2009. Producing wrong
764 data without doing anything obviously wrong!, in: Proceedings of the 14th
765 International Conference on Architectural Support for Programming Languages
766 and Operating Systems (ASPLOS '09). doi:10.1145/1508244.1508275.

767 Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P.F., 2010. Evaluating the
768 accuracy of Java profilers, in: Proceedings of the 31st ACM SIGPLAN Conference
769 on Programming Language Design and Implementation (PLDI '10), pp. 187–197.
770 doi:10.1145/1806596.1806618.

771 Nadi, S., Krüger, S., Mezini, M., Bodden, E., 2016. Jumping through hoops, in:
772 Proceedings of the 38th International Conference on Software Engineering (ICSE
773 '16). doi:10.1145/2884781.2884790.

774 Naser, A., Sadeghi Lahijani, M., Wu, C., Gavahi, M., Hoang, V.T., Wang, Z., Yuan,
775 X., 2020. Performance evaluation and modeling of cryptographic libraries for
776 MPI communications. arXiv preprint arXiv:2010.06139.

777 Nistor, A., Jiang, T., Tan, L., 2013. Discovering, reporting, and fixing performance
778 bugs, in: Proceedings of the 10th Working Conference on Mining Software
779 Repositories (MSR '13). doi:10.1109/MSR.2013.6624035.

780 Ogasawara, T., Komatsu, H., Nakatani, T., 2006. EDO: Exception-directed
781 optimization in Java. ACM Transactions on Programming Languages and
782 Systems 28. doi:10.1145/1111596.1111598.

783 de Pádua, G.B., Shang, W., 2017. Studying the prevalence of exception handling
784 anti-patterns, in: Proceedings of the 25th International Conference on Program
785 Comprehension (ICPC '17), pp. 328–331. doi:10.1109/ICPC.2017.1.

786 Parravicini, A., Mueller, T., 2021. The cost of speculation: Revisiting overheads in
787 the V8 JavaScript engine, in: 2021 IEEE International Symposium on Workload
788 Characterization (IISWC). doi:10.1109/IISWC53511.2021.00013.

789 Rahaman, S., Xiao, Y., Afrose, S., Shaon, F., Tian, K., Frantz, M., Kantarcioglu,
790 M., Yao, D.D., 2019. CryptoGuard: High precision detection of cryptographic
791 vulnerabilities in massive-sized Java projects, in: Proceedings of the 2019 ACM
792 SIGSAC Conference on Computer and Communications Security (CCS '19), pp.
793 2455–2472. doi:10.1145/3319535.3345659.

794 Selakovic, M., Glaser, T., Pradel, M., 2017. An actionable performance profiler
795 for optimizing the order of evaluations, in: Proceedings of the 26th ACM
796 SIGSOFT International Symposium on Software Testing and Analysis (ISSTA
797 '17). doi:10.1145/3092703.3092716.

798 Selakovic, M., Pradel, M., 2016. Performance issues and optimizations in JavaScript:

799 An empirical study, in: Proceedings of the 38th International Conference on
800 Software Engineering (ICSE '16), pp. 61–72. doi:10.1145/2884781.2884829.

801 Sheffer, Y., Hardt, D., Jones, M.B., 2020. JSON Web Token Best Current Practices.
802 Request for Comments RFC 8725. Internet Engineering Task Force. doi:10.174
803 87/RFC8725.

804 Sikeridis, D., Kampanakis, P., Devetsikiotis, M., 2020. Assessing the overhead of
805 post-quantum cryptography in TLS 1.3 and SSH, in: Proceedings of the 16th
806 International Conference on Emerging Networking EXperiments and Technologies
807 (CoNEXT '20). doi:10.1145/3386367.3431305.

808 Staicu, C.A., Pradel, M., 2018. Freezing the web: A study of ReDoS vulnerabilities
809 in JavaScript-based web servers, in: Proceedings of the 27th USENIX Security
810 Symposium, pp. 361–376.

811 Traini, L., Cortellessa, V., Di Pompeo, D., Tucci, M., 2023. Towards effective
812 assessment of steady state performance in Java software: Are we there yet?
813 Empirical Software Engineering 28. doi:10.1007/s10664-022-10247-x.

814 Xu, B., Jia, S., Lin, J., Zheng, F., Ma, Y., Liu, L., Gu, X., Song, L., 2023.
815 JWTKey: Automatic cryptographic vulnerability detection in JWT applications,
816 in: Computer Security – ESORICS 2023, Springer. doi:10.1007/978-3-031-5
817 1479-1_14.

818 Yang, J., Wang, E., Chen, J., Wang, Q., Zhang, Y., Duan, H., Xie, W., Wang,
819 B., 2026. Token time bomb: Evaluating JWT implementations for vulnerability
820 discovery, in: Proceedings of the Network and Distributed System Security
821 Symposium (NDSS '26).

822 Zerouali, A., Mens, T., Robles, G., Gonzalez-Barahona, J.M., 2019. On the
823 relation between outdated Docker containers, severity vulnerabilities, and bugs,
824 in: Proceedings of the 26th IEEE International Conference on Software Analysis,
825 Evolution and Reengineering (SANER '19), pp. 491–501. doi:10.1109/SANER.
826 2019.8668013.

827 Zhu, X., She, G., Xue, B., Zhang, Y., Zhang, Y., Zou, X.K., Duan, X., He, P.,
828 Krishnamurthy, A., Lentz, M., Zhuo, D., Mahajan, R., 2023. Dissecting overheads
829 of service mesh sidecars, in: Proceedings of the 2023 ACM Symposium on Cloud
830 Computing (SoCC '23). doi:10.1145/3620678.3624652.

Table 11: Summary of findings, evidence and implications, by research question and by the stakeholder who can act on each.

RQ	Finding	Evidence	Implication (who acts)
RQ1	The signature is 2.208 μ s of a 29.92 μ s call; a discarded parse is 18.04 μ s (60%).	Table 3, Fig. 3; profiler	Profile before relocating a “cryptographic” cost; attribution by intuition was wrong twice here (<i>performance engineers</i>).
RQ2	The parse costs 394.4 μ s on OpenSSL 3.0.16 and 8.447 μ s on 3.5.8, issued from C; Ubuntu 24.04’s own Node.js pays the OpenSSL 3.0 price.	Tables 5, 6; Fig. 4	Record the base image and bundled OpenSSL with every performance figure; treat base-image bumps as performance changes (<i>developers, DevOps</i>).
RQ3	In a containerised service on OpenSSL 3.0, validation is 53% of the application’s CPU per request; none of the 327 sampled call sites pre-parses its key.	Table 7; Figs. 5, 6	Pre-parse key material once at startup, a one-line change (Figure 10) (<i>application developers</i>).
RQ4	Of seven libraries in four languages only one uses a failing parse; two pay for per-call key handling, and only the failing parse’s cost changes by an order of magnitude with the OpenSSL release; in 64+164 popular npm packages only <code>jsonwebtoken</code> uses the failing parse, in <code>verify()</code> and in <code>sign()</code> , where a string secret costs 436 μ s per call on Ubuntu 24.04’s Node.js.	Table 8, Fig. 7; §7.1	Do not use a failing parse as a type test; resolve keys once, inspect then dispatch, or type the key (<i>library authors</i>).
RQ5	The parse is load-bearing for a key-confusion defence: the naive optimisation fails 2 library tests. Dispatching on the key material alone passes all 511 on 4 runtimes and removes the parse for every token, forged ones included.	Table 9; Figs. 8, 9	Adopt the key-material fix; review performance changes to verification code as security changes and run the key-confusion tests (<i>maintainers, reviewers</i>).